PROJECT REPORT

ON

"Student Performance Analysis and Prediction System"

Submitted By:

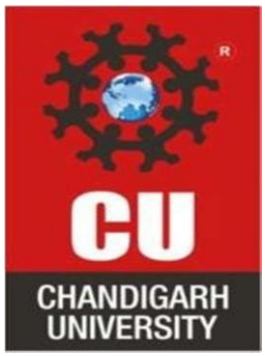
Sreenivas Bharadwaj

UID- 26MCA20094

Under The Guidance of:

Dr. Gagandeep kaur

October 2026



University Institute of Computing

Chandigarh University,

Mohali, Punjab

CERTIFICATE

This is to certify that Mr. Sreenivas Bharadwaj (UID- 26MCA20099) have successfully completed the project title **"Student Performance Analysis and Prediction System"** at University Institute of Computing under my supervision and guidance in the fulfilment of requirements of first semester, Master of Computer Application- Specialization in General. Of Chandigarh University, Mohali, Punjab.

Dr. Amanpreet Kaur Sandhu

Head of the Department

University Institute of Computing

Dr. Gagandeep kaur

Project Guide Supervisor

University Institute of Computing

ACKNOWLEDGEMENT

We deem it a pleasure to acknowledge our sense of gratitude to our project guide **Dr. Gagandeep kaur** under whom we have carried out the project work.
His incisive and objective guidance and timely advice encouraged us with constant flow of energy to continue the work.

We wish to reciprocate in full measure the kindness shown by Dr. **Amanpreet Kaur Sandhu** (H.O.D, University Institute of Computing) who inspired us with his valuable suggestions in successfully completing the project work.

We shall remain grateful to **Dr. Manisha Malhotra,** Additional Director, University Institute of Technology, for providing us a strong academic atmosphere by enforcing strict discipline to do the project work with utmost concentration and dedication.

Finally, we must say that no height is ever achieved without some sacrifices made at some end and it is here where we owe our special debt to our parents and our friends for showing their generous love and care throughout the entire period.

Date: 27.10.2026

Place: Chandigarh University, Mohali, Punjab

Sreenivas Bharadwaj,

UID- 26MCA20094

ABSTRACT

Student performance analysis plays an important role in understanding students' academic progress and identifying areas that require improvement. The project titled **"Student Performance Analysis and Prediction System"** focuses on managing student records, analyzing academic performance, and estimating student scores using Python and web technologies.

The project uses Python and the Flask framework for backend processing, MySQL for storing and managing student records, and HTML and CSS for developing the web interface. The system maintains information such as student ID, name, roll number, class, attendance percentage, subject-wise marks, and daily study hours.
The system provides features to add, update, search, view, and delete student records. Average marks are calculated to evaluate individual performance, while subject-wise analysis helps compare academic results. A dashboard displays important information, including the total number of students, class average, pass rate, and students who may require additional academic support. A score estimation feature uses previous academic performance, attendance, and study hours to calculate an indicative score and performance category.

The project demonstrates how Python, web development technologies, and database management can be combined to simplify academic record management and present useful performance summaries. The current prediction feature uses a rule-based calculation and is intended for educational demonstration rather than accurate forecasting. The system can be further enhanced by integrating machine-learning algorithms, graphical reports, and secure user authentication.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1.	Abstract	iv
2.	Introduction	1
3.	Problem Statement	1
4.	Objectives of the Project	1
5.	Scope of the Project	2
6.	System Requirements	2
7.	Tools and Technologies Used	3
8.	Dataset description	3
9.	Implementation and Python Code	4
10.	Output and Screenshot	5-7
11.	Limitations	8
12.	Future Scope	9
13.	Conclusion	9
14.	References	9

INTRODUCTION

The **Student Performance Analysis and Prediction System** is a web-based application developed to manage student records, analyse academic performance, and estimate students' scores. The main objective of this project is to simplify academic record management and help identify students who may need additional academic support.

The system is developed using Python and Flask for backend processing, MySQL for database management, and HTML and CSS for designing the user interface. It allows users to add, view, search, update, and delete student records, including attendance, subject-wise marks, and daily study hours.

The application calculates average marks, analyse subject-wise performance, and displays important information through a dashboard. It also provides a basic score estimation feature based on available academic data. Overall, this project demonstrates how programming and database technologies can be used to organize student information, reduce manual effort, and support better understanding of academic progress.

PROBLEM STATEMENT

Managing student academic records manually can be time-consuming and may lead to errors in data entry, calculation, and performance evaluation. It can also be difficult for teachers to compare students' results, monitor attendance, and identify students who require additional academic support.

The **Student Performance Analysis and Prediction System** aims to address these problems by providing a computerized platform to store, manage, and analyze student information. The system evaluates academic performance using subject-wise marks, attendance percentage, and daily study hours. It generates performance summaries and provides basic score estimates to help users understand students' academic progress more effectively.

OBJECTIVES OF THE PROJECT

The main objective of the **Student Performance Analysis and Prediction System** is to simplify student record management and analyze academic performance efficiently.
The major objectives of the project are:
Student Record Management: To add, view, search, update, and delete student information easily.
Academic Performance Analysis: To calculate average marks and evaluate students' academic progress.
Attendance Monitoring: To consider attendance percentage as an important indicator of student performance.
Performance Identification: To identify students who may require additional academic support.
Score Estimation: To estimate student scores using available academic information, including marks, attendance, and study hours.
Dashboard and Reports: To display important statistics, such as total students, class average, and pass rate, in an organized manner.
Efficient Data Management: To use MySQL for storing and managing student records systematically.
Overall, the project aims to reduce manual effort, improve the organization of academic information, and support better understanding of student performance.

SCOPE OF THE PROJECT

The **Student Performance Analysis and Prediction System** aims to simplify student record management and academic performance evaluation. It helps users organize academic information and understand students' progress through a web-based platform.
The scope of the project includes:

- **Student Record Management:** Maintaining and updating student information.
- **Performance Analysis:** Calculating average marks and comparing subject-wise results.
- **Attendance Monitoring:** Recording attendance percentages.
- **Score Estimation:** Estimating student performance using available academic data.
- **Dashboard:** Displaying class average, total students, and pass rate.
- **Academic Support:** Identifying students who may need improvement.

The system can be further enhanced with machine-learning algorithms, graphical reports, and personalized learning recommendations.

SYSTEM REQUIREMENTS

1. Hardware Requirements

- **Processor:** Intel or AMD processor, or equivalent.
- **RAM:** Minimum 4 GB recommended.
- **Storage:** At least 500 MB of free space.
- **Input Devices:** Keyboard and mouse.
- **Output Device:** Monitor or display.

1. Software Requirements

- **Operating System:** Windows 10/11, Linux, or macOS.
- **Programming Language:** Python 3.x.
- **Web Framework:** Flask.
- **Database Management System:** MySQL.
- **Code Editor:** Visual Studio Code or any Python-supported IDE.
- **Web Browser:** Google Chrome or Microsoft Edge.
- **Execution Environment:** Python interpreter and terminal.

1. Python Libraries

- **Flask:** Used to develop the web application and handle requests between the user interface and backend.
- **MySQL Connector:** Used to connect the Python application with the MySQL database and manage student records.
- **HTML:** Used to structure the web pages and display student information.
- **CSS:** Used to design the interface and improve the appearance of the application.

TOOLS AND TECHNOLOGIES USED

The Student Performance Analysis and Prediction System uses the following tools and technologies for development:
Python: Used for backend programming, processing student data, calculating average marks, and estimating performance scores.
Flask: A Python web framework used to develop the web application and manage communication between the frontend and backend.
MySQL: Used to store, retrieve, update, and manage student academic records.
HTML: Used to create the structure of web pages, including forms, tables, and dashboards.
CSS: Used to style the web pages and provide a user-friendly interface.
MySQL Connector: Used to establish a connection between Python and the MySQL database.
Visual Studio Code: Used to write, edit, and manage the project code.
Web Browser: Google Chrome or Microsoft Edge is used to access and test the application locally.

DATASET DESCRIPTION

The **Student Performance Analysis and Prediction System** uses student academic records stored in a MySQL database. The dataset contains information required to analyze students' academic performance and estimate their scores. The records can be entered and managed through the web application.

The dataset includes the following attributes:

Student ID: A unique identifier assigned to each student.
Student Name: The name of the student.
Roll Number: The student's academic roll number.
Class: The class or academic group of the student.
Attendance Percentage: The percentage of classes attended.
Subject-wise Marks: Marks obtained in different subjects.
Daily Study Hours: The average number of hours spent studying per day.

The system uses these attributes to calculate average marks, classify performance, compare academic results, and generate estimated scores. The dataset helps organize student information and provides a basis for understanding academic progress.

IMPLEMENTATION AND PYTHON CODE

The Student Performance Analysis and Prediction System is implemented using Python, Flask, MySQL, HTML, and CSS. Python handles the application logic, Flask manages web requests, and MySQL stores student records. The frontend provides forms, tables, and a dashboard to display academic information.

8.1 Implementation Steps

Database Setup: Create a MySQL database to store student details, attendance, subject-wise marks, and study hours.
Backend Development: Use Python and Flask to handle student records and perform calculations.
Frontend Development: Design web pages using HTML and CSS.
Performance Analysis: Calculate average marks and classify student performance.
Score Estimation: Estimate a student's score using available academic information.
Testing: Run the application locally and verify that database operations and analysis work correctly.

8.2 Python Code

The following code demonstrates the basic logic used to calculate average marks and classify student performance.

```
def analyze_performance(marks, attendance, study_hours):
    average = sum(marks) / len(marks)

    if average >= 75:
        category = "Excellent"
    elif average >= 60:
        category = "Good"
    elif average >= 40:
        category = "Average"
    else:
        category = "Needs Improvement"

    print("Average Marks:", round(average, 2))
    print("Attendance:", attendance, "%")
    print("Daily Study Hours:", study_hours)
    print("Performance Category:", category)


# Example student data
marks = [78, 85, 72, 80]
attendance = 90
study_hours = 3

analyze_performance(marks, attendance, study_hours)
```

1. Login Page

Users sign in with a username and password. Passwords are verified against a stored hash, and unauthenticated visitors are redirected to this page. A demo account (admin / admin123) is created on first run.

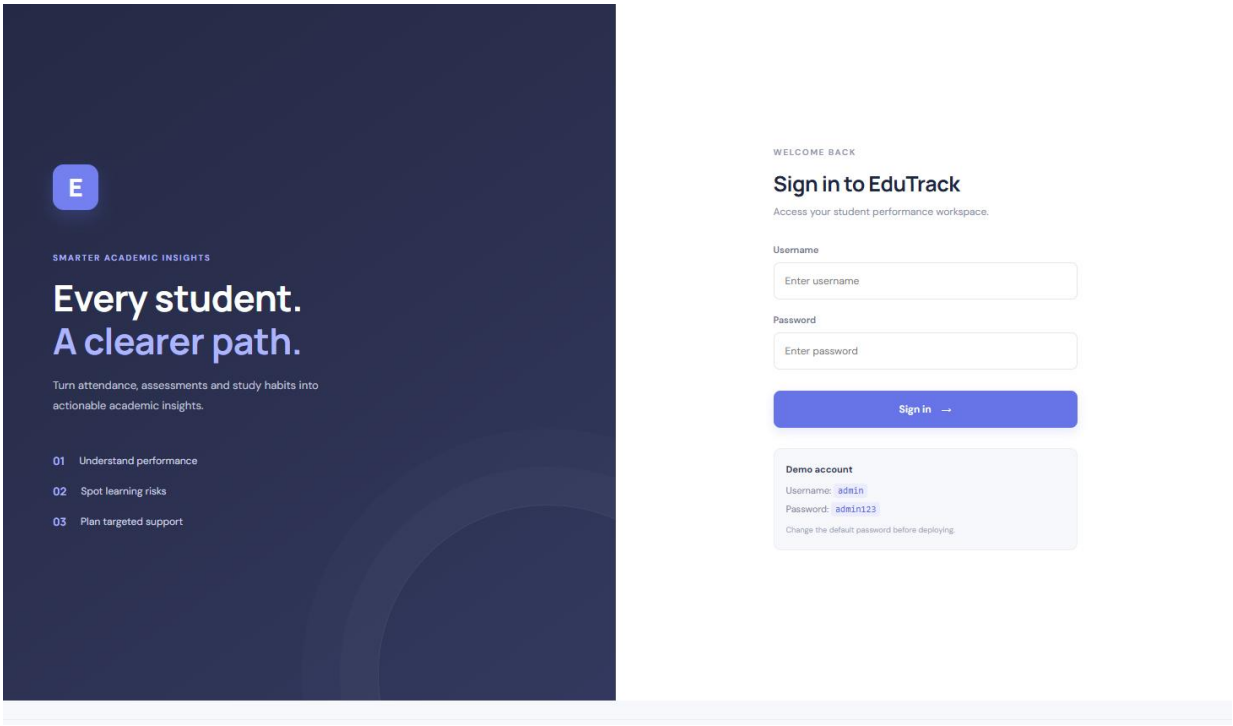


Figure 1: Login page

2. Dashboard

The dashboard gives a class overview: total students, class average (68.5), students at risk, subjects tracked, a performance chart and top performers.

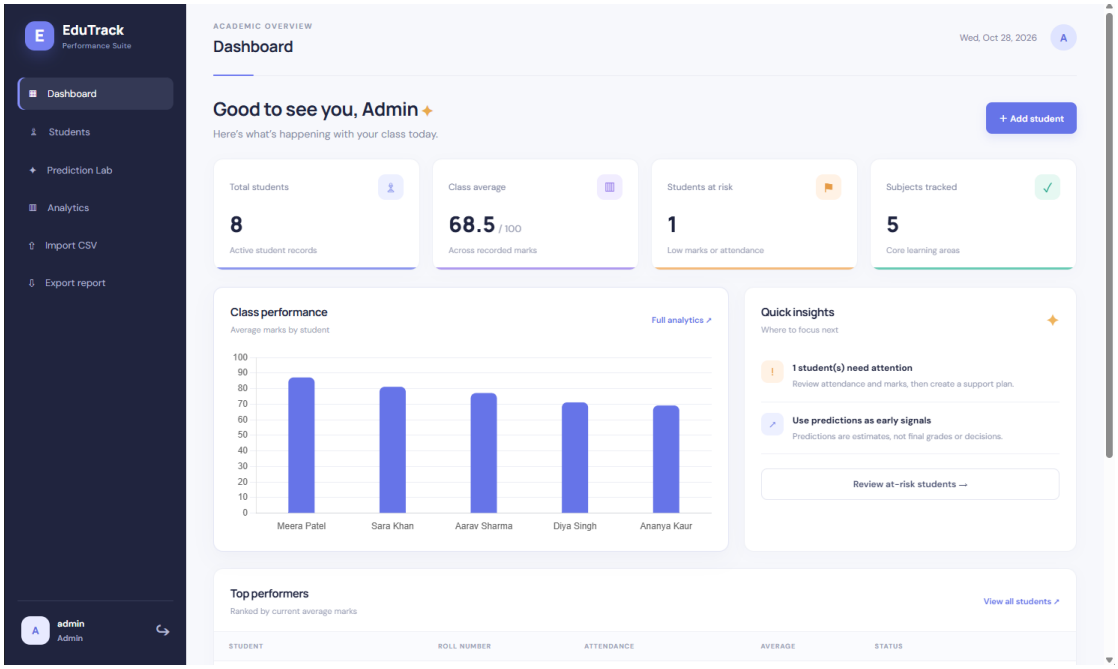
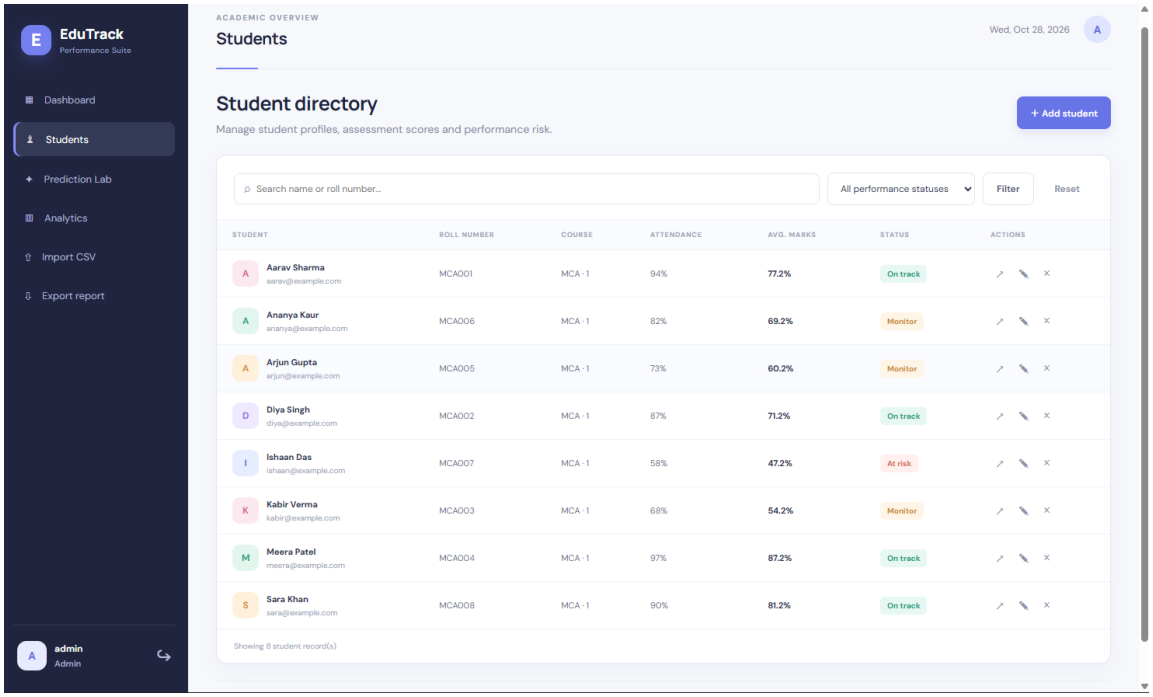


Figure 2 : Dashboard with class overview

3 . Student Directory

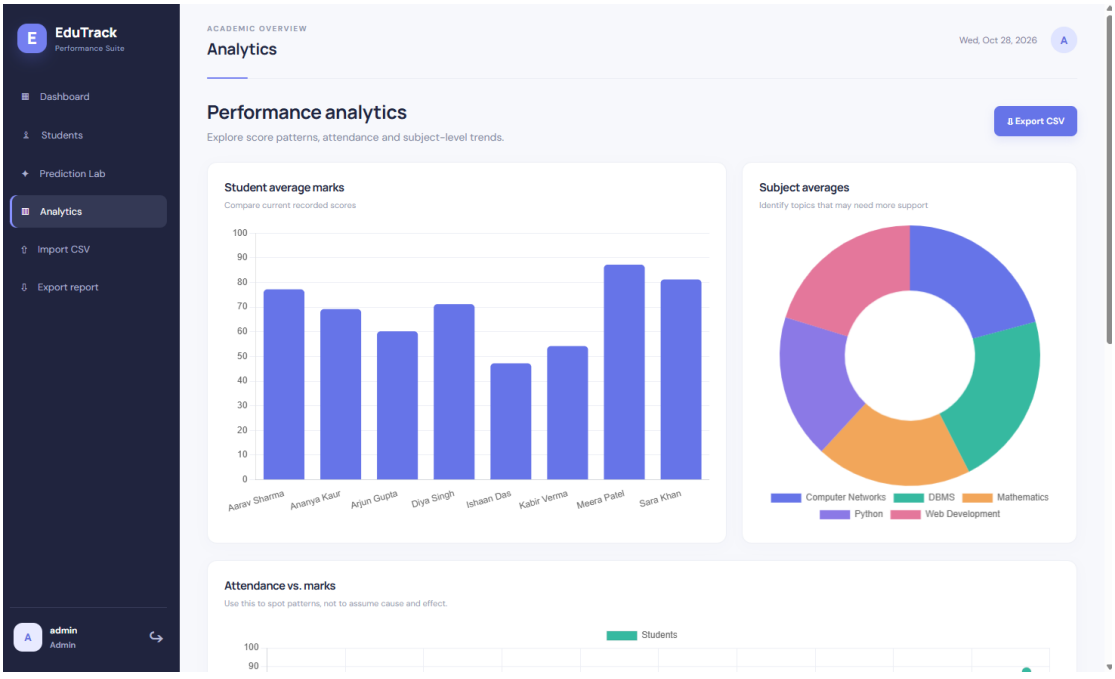
Students are listed with attendance, average marks and a status label (On track, Monitor, At risk). Teachers can search by name or roll number, filter by status, and view, edit or delete a record. Inputs are validated: percentages must be 0–100 and study hours 0–24.



3: Student directory with search and filter

4 . Analytics

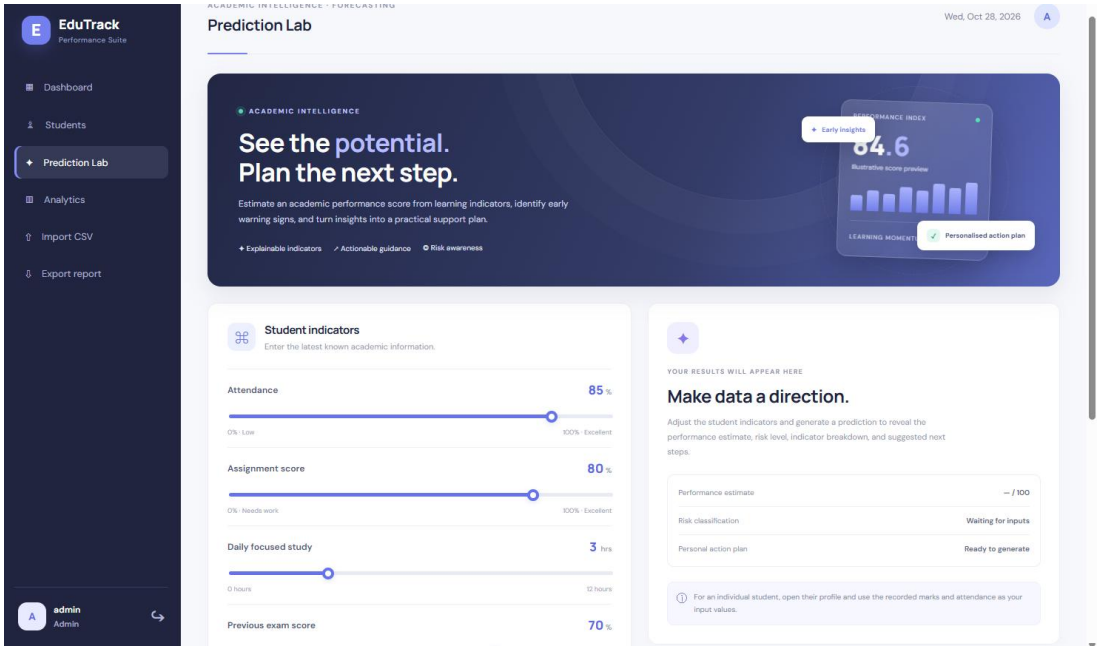
The analytics page uses Chart.js to show each student's average marks, the average per subject, and attendance versus marks to help spot patterns.



.4: Performance analytics

5 Prediction Lab

The teacher sets attendance, assignment score, daily study hours, previous exam score and current average. On running the prediction the page shows the estimated score, risk classification, indicator breakdown and a tailored action plan.



5: Prediction Lab

Prediction Model

When scikit-learn is available and at least six students have marks, a RandomForestRegressor (100 trees) is trained on the class records; otherwise a transparent weighted formula is used. Study hours are scaled so that 5 h/day = 100 points.

```
indicator = 0.25*attendance + 0.20*assignment + 0.15*min(study_hours*20,100)
        + 0.20*previous_score + 0.20*current_average
```

Risk level	Rule
High	predicted score < 50 or attendance < 65 %
Medium	predicted score < 70 or attendance < 75 %
Low	otherwise

Results :

Stored values for the eight demo students and the Random Forest predictions are shown below. The class average is 68.5. Ishaan Das (MCA007) is correctly flagged High risk (58 % attendance, 0.8 study hours/day), while Meera Patel (MCA004) has the highest predicted score.

Roll	Name	Att.%	Assg.	Hrs	Prev.	Avg	Pred.	Risk	Grade
MCA001	Aarav Sharma	94	88	3.5	78	77.2	82.4	Low	B
MCA002	Diya Singh	87	81	2.8	72	71.2	75.2	Low	B
MCA003	Kabir Verma	68	62	1.2	55	54.2	56.3	Medium	C
MCA004	Meera Patel	97	93	4	88	87.2	87.3	Low	A
MCA005	Arjun Gupta	73	70	2	61	60.2	61.8	Medium	C
MCA006	Ananya Kaur	82	79	2.6	70	69.2	73.3	Low	B
MCA007	Ishaan Das	58	52	0.8	48	47.2	51.4	High	D
MCA008	Sara Khan	90	86	3.2	82	81.2	82.0	Low	B

Testing

Each feature was exercised with valid, invalid and boundary inputs: correct and wrong login, access without login, duplicate roll number, attendance above 100, study hours above 24, marks above 100, blank prediction fields, CSV without the name column, student deletion with cascading marks, and invalid URLs (custom 404 page). The application handled each case with the expected message or behavior.

LIMITATIONS

- The Student Performance Analysis and Prediction System has certain limitations:
- **Data Dependency:** The accuracy of the analysis depends on the correctness of the entered student data.
- **Basic Prediction:** The system uses rule-based calculations rather than a trained machine-learning model.
- **Limited Parameters:** Performance analysis is based mainly on marks, attendance, and daily study hours.
- **Manual Data Entry:** Student information must be entered and updated by the user.
- **Local Access:** The application runs locally on the computer and is not available online by default.
- **Limited Scalability:** The current version is designed for basic academic record management and may require improvements to support large institutions.

FUTURE SCOPE

The Student Performance Analysis and Prediction System can be improved in several ways in the future:

- **Machine Learning Integration:** Implement machine-learning algorithms to improve student performance prediction.
- **Data Visualization:** Add graphical charts and reports to represent academic performance clearly.
- **Online Deployment:** Host the application on a web server so it can be accessed from different locations.
- **User Authentication:** Introduce secure login facilities for teachers and administrators.
- **Automated Notifications:** Send alerts when students show low marks or attendance.
- **Personalized Recommendations:** Suggest study plans and improvement strategies based on individual performance.
- **Advanced Reporting:** Generate downloadable student performance reports for academic evaluation.

These enhancements can make the system more efficient, accessible, and useful for academic institutions.

CONCLUSION

The **Student Performance Analysis and Prediction System** provides an effective way to manage student records and analyze academic performance using Python, Flask, and MySQL. The system helps users maintain student information, calculate average marks, monitor attendance, and identify students who may require additional academic support.

The project demonstrates how web technologies and database management can reduce manual effort and organize academic information efficiently. Although the current score estimation feature is based on simple rule-based calculations, the system provides a foundation for future improvements using machine learning, graphical reports, and online deployment. Overall, the project helps simplify academic record management and supports better understanding of student performance.

REFERENCES

Python Software Foundation, “Python Documentation.” Available: https://docs.python.org/3/

Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/

Oracle, “MySQL Reference Manual.” Available: https://dev.mysql.com/doc/

MDN Web Docs, “HTML: HyperText Markup Language.” Available: https://developer.mozilla.org/en-US/docs/Web/HTML

MDN Web Docs, “CSS: Cascading Style Sheets.” Available: https://developer.mozilla.org/en-US/docs/Web/CSS

W3Schools, “Python Tutorial.” Available: https://www.w3schools.com/python/

W3Schools, “SQL Tutorial.” Available: https://www.w3schools.com/sql/